

第五次编程实践课

软件工程与计算 I · 2026 · 南京大学

| Week 5 · 线性数据结构 · AI 编程进阶

今日安排

知识讲解

Java 线性数据结构

LinkedList · Stack · Queue · Deque

AI实践 AI 编程进阶

项目级 AI 技巧 · AI 辅助代码质量管理

大作业 Project 03 答案讲解 + Project 04 布置

AST 解析器答案解析 · Eval 求值器任务讲解

当堂练习

练习 共 2 题，独立完成

- 练习 1: 用 Stack 实现括号匹配
- 练习 2: 用 Queue 模拟任务调度

目标：掌握线性数据结构的使用场景，建立对 Stack/Queue 的直觉

知识讲解

线性数据结构 · AI 编程进阶 · 大作业 Project 03/04

线性数据结构概览

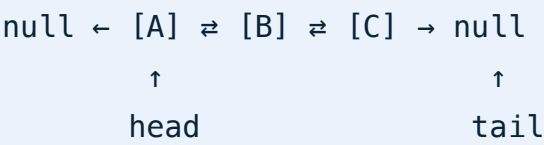
上次课学了 **Array** 和 **ArrayList**，本节介绍 Java 集合框架中的其他线性结构：

数据结构	底层实现	特点	典型场景
ArrayList	动态数组	随机访问 $O(1)$ ，尾部增删 $O(1)$	需要按下标访问
LinkedList	双向链表	头尾增删 $O(1)$ ，随机访问 $O(n)$	频繁头部插入/删除
Stack	数组	后进先出 (LIFO)	括号匹配、调用栈
Queue	链表/数组	先进先出 (FIFO)	任务调度、BFS
Deque	数组/链表	两端都可增删	双端操作、滑动窗口

Java 集合框架的优势：统一的接口—— **List**、**Queue**、**Deque**，底层实现可以切换。

LinkedList — 双向链表

结构示意



每个节点持有 `prev` 和 `next` 两个指针。

核心操作

```

LinkedList<String> list = new LinkedList<>();
list.addFirst("A");      // [A]
list.addLast("B");       // [A, B]
list.add(1, "X");        // [A, X, B]
list.removeFirst();      // [X, B]
list.removeLast();       // [X]
list.get(0);             // "X"

```

ArrayList vs LinkedList

操作	ArrayList	LinkedList
按下标访问 <code>get(i)</code>	O(1)	O(n)
尾部追加 <code>add(e)</code>	O(1) 均摊	O(1)
头部插入 <code>add(0,e)</code>	O(n) 搬移	O(1)
中间插入 <code>add(i,e)</code>	O(n)	O(n)*
遍历	缓存友好	缓存不友好

* 找到位置 O(n) + 插入 O(1)

选择指南：大多数场景用 ArrayList；
只有频繁在**头部**增删时才考虑 LinkedList。

LinkedList 作为 Queue 和 Deque

LinkedList 同时实现了 List、Queue、Deque 三个接口：

```
// 作为 Queue 使用 (FIFO)
Queue<String> queue = new LinkedList<>();
queue.offer("任务1");    // 入队 (尾部)
queue.offer("任务2");    // 入队 (尾部)
queue.poll();            // 出队 (头部) → "任务1"

// 作为 Deque 使用 (双端队列)
Deque<String> deque = new LinkedList<>();
deque.offerFirst("A");    // 头部入队
deque.offerLast("B");     // 尾部入队
deque.pollFirst();        // 头部出队 → "A"
deque.pollLast();         // 尾部出队 → "B"
```

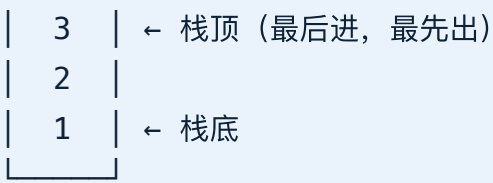
接口编程：声明类型用接口（Queue<>），具体实现可替换——这就是多态。

Stack — 栈（后进先出）

栈的操作

```
Stack<Integer> stack = new Stack<>();
stack.push(1);      // 压入 [1]
stack.push(2);      // 压入 [1, 2]
stack.push(3);      // 压入 [1, 2, 3]
stack.peek();       // 查看栈顶 → 3
stack.pop();         // 弹出 → 3    [1, 2]
stack.pop();         // 弹出 → 2    [1]
stack.isEmpty();     // false
stack.size();        // 1
```

栈的结构



典型应用场景

场景	说明
括号匹配	左括号入栈，右括号匹配弹出
函数调用栈	JVM 用栈管理每次函数调用
表达式求值	逆波兰表达式计算
浏览器历史	后退 = pop, 前进 = 另一个栈
撤销操作	Ctrl+Z = 弹出上一个操作

现代替代

```
// 推荐用 Deque 替代 Stack
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.pop();
```

`Stack` 是遗留类（线程安全但慢），新代码建议用 `ArrayDeque`。

Stack 实战 — 括号匹配

问题：判断括号是否匹配。"((()))" 匹配，"((()" 不匹配。

```
static boolean isBalanced(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);           // 左括号入栈
        } else if (c == ')' || c == ']' || c == '}') {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if (!matches(top, c)) return false;
        }
    }
    return stack.isEmpty();
}
```

辅助方法

```
static boolean matches(char open, char close) {
    return (open == '(' && close == ')')
        || (open == '[' && close == ']')
        || (open == '{' && close == '}');
}
```

算法思路

步骤	操作
遇左括号	push 入栈
遇右括号	pop 并检查是否匹配
遍历结束	栈为空 = 全部匹配

与大作业的联系：Parser 中 `parseList` 的左右括号匹配，本质就是隐式的栈操作！

Queue — 队列（先进先出）

队列的操作

```
Queue<String> q = new LinkedList<>();
q.offer("A");      // 入队 [A]
q.offer("B");      // 入队 [A, B]
q.offer("C");      // 入队 [A, B, C]
q.peek();          // 查看队首 → "A"
q.poll();          // 出队 → "A"    [B, C]
q.poll();          // 出队 → "B"    [C]
q.isEmpty();       // false
```

队列的结构



两组 API 的区别

操作	抛异常版	返回特殊值版
入队	<code>add(e)</code> 满则异常	<code>offer(e)</code> 满则 false
出队	<code>remove()</code> 空则异常	<code>poll()</code> 空则 null
查看	<code>element()</code> 空则异常	<code>peek()</code> 空则 null

推荐使用 `offer` / `poll` / `peek` ——
不抛异常，代码更健壮。

典型场景

- BFS 搜索：层序遍历
- 任务调度：先来先服务
- 消息队列：生产者-消费者模型

Deque — 双端队列

Deque (Double-Ended Queue)：两端都可以入队和出队。

```
Deque<String> deque = new ArrayDeque<>(); // 推荐实现

deque.offerFirst("B"); // 头部入 [B]
deque.offerLast("C"); // 尾部入 [B, C]
deque.offerFirst("A"); // 头部入 [A, B, C]

deque.peekFirst(); // 查看头 → "A"
deque.peekLast(); // 查看尾 → "C"

deque.pollFirst(); // 头部出 → "A" [B, C]
deque.pollLast(); // 尾部出 → "C" [B]
```

Deque 可以同时充当 Stack 和 Queue

用途	入	出
当 Stack 用	push = offerFirst	pop = pollFirst
当 Queue 用	offer = offerLast	poll = pollFirst

ArrayDeque：基于循环数组，比 LinkedList 和 Stack 都快——**首选实现**。

ArrayDeque — 底层原理

循环数组结构

容量 8 的 ArrayDeque:

索引: 0 1 2 3 4 5 6 7
[] [] [C] [D] [E] [] [] []
 ↑ ↑
 head tail

offerFirst("B"): head 左移
[] [B] [C] [D] [E] [] [] []
 ↑ ↑
 head tail

offerLast("F"): tail 右移
[] [B] [C] [D] [E] [F] [] []
 ↑ ↑
 head tail

- **head** 和 **tail** 在数组中**循环移动**（到头了就绕回 index 0）
- 满了自动 **2 倍扩容**（类似 ArrayList）
- 没有链表节点开销，**缓存友好**

循环数组 = 用取模实现"首尾相连"，头尾操作都是 $O(1)$ 。

ArrayDeque — 使用方式与性能对比

当 Stack 使用

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);      // [10]
stack.push(20);      // [20, 10]
stack.push(30);      // [30, 20, 10]
stack.peek();        // 30 (查看栈顶)
stack.pop();          // 30 (弹出栈顶)
```

当 Queue 使用

```
Queue<String> queue = new ArrayDeque<>();
queue.offer("任务A"); // [任务A]
queue.offer("任务B"); // [任务A, 任务B]
queue.poll();         // "任务A" (先进先出)
```

为什么优于 Stack 和 LinkedList?

对比	ArrayDeque	Stack / LinkedList
线程安全开销	无	Stack 有 (synchronized)
内存开销	连续数组	每个节点额外指针
缓存命中率	高	低 (节点分散)

注意事项

- 不允许 null 元素 (null 用作哨兵值)
- 非线程安全 (多线程需外部同步)
- 不支持按下标访问 (不是 List)

一句话总结: 需要 Stack 或 Queue 时, 默认选 `ArrayDeque` 。

线性数据结构选择指南

- 需要按下标随机访问?
- └ 是 → ArrayList
 - └ 否 → 需要什么操作模式?
 - └ 后进先出 (LIFO) → ArrayDeque (当 Stack 用)
 - └ 先进先出 (FIFO) → ArrayDeque 或 LinkedList (当 Queue 用)
 - └ 两端增删 → ArrayDeque (当 Deque 用)
 - └ 频繁头部插入/删除 → LinkedList

使用方式	推荐写法
动态数组	List<E> list = new ArrayList<>()
栈	Deque<E> stack = new ArrayDeque<>()
队列	Queue<E> queue = new ArrayDeque<>()
双端队列	Deque<E> deque = new ArrayDeque<>()

核心思想：用接口声明类型，用实现创建对象——面向接口编程。

Collections.unmodifiableList — 问题与解决

问题：可变列表的风险

```
ArrayList<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");

printAll(names); // 把列表传给别人

// 对方偷偷修改了你的列表!
static void printAll(List<String> list) {
    list.clear(); // 灾难!
}
```

调用者可以随意修改你的数据——
多人协作中很容易引发 Bug。

解决：返回不可变视图

```
import java.util.Collections;

List<String> safe =
    Collections.unmodifiableList(names);

safe.get(0);           // "Alice" ✓ 可读
safe.add("Carol");     // 抛出异常! ✗
// UnsupportedOperationException
```

操作	结果
get / size / 遍历	正常使用
add / remove / set	抛异常

不可变视图：只读不写，保护原始数据。

unmodifiableList — 在大作业中的应用

LList 的防御性设计

LList 的 content 就使用了不可变列表：

```
public class LList extends LObject {
    public final List<LObject> content;

    public LList(List<LObject> lst) {
        // 先复制，再包装为不可变
        this.content = Collections
            .unmodifiableList(
                new ArrayList<>(lst));
    }
}
```

为什么先复制再包装？

```
// 不复制 → 调用者改原 lst, content 跟着变!
this.content = Collections.unmodifiableList(lst);

// 先复制 → 调用者改原 lst 不影响 content
this.content = Collections
```

为什么这样设计？

好处	说明
安全	防止外部代码篡改 AST 节点
可预测	equals / hashCode 永远一致
线程安全	不可变对象天然线程安全

不可变设计三步法

步骤	做法
防御性复制	new ArrayList<>(lst)
包装不可变	unmodifiableList(...)
声明 final	public final List<> content

防御性编程——假设外部调用者都可能犯错，在设计时堵住修改入口。

AI4Coding 进阶

| 项目级 AI 技巧 · AI 辅助代码质量

AI4Coding — 从"写 10 行"到"写 500 行"

核心问题 AI 能写出 10 行的 Fibonacci，但 500 行的项目怎么办？

简单项目 vs 项目级项目

维度	简单项目	项目级项目
代码行数	< 50 行	500+ 行
文件数	1 个	10+ 个
AI 提示词	"写个排序"	完整的设计说明书
核心挑战	算法正确	架构一致 + 跨文件协调

本节要点

项目级 AI 技巧

- 上下文注入与约束绑定
- 推理先行与任务分解
- 跨文件感知与回归防护

AI 辅助代码质量

- Debug 系统化四步法
- AI 生成测试用例
- AI 辅助代码审查

区别不在于 AI 更强，
而在于你提问的方式。

项目级技巧① — 上下文注入与约束绑定

在对话开始时注入项目约束，让 AI 成为"项目规范守护者"

[项目约束注入]

项目名: Lambda Calculus 解释器

技术栈: Java 11 + Maven + JUnit 4

架构: Lexer → Parser → Evaluator 流水线

编码规范: 每个类有 Javadoc 注释

在整个对话中，请始终遵守以上约束。

冲突提醒技巧

"如果你的实现与上述约束有冲突，
请先说明冲突点，再给出符合约束的方案。"

没有约束注入，AI 可能用 Python 回答你的 Java 问题，或者引入你不需要的库。

项目级技巧② — 推理先行与任务分解

不要让 AI 直接写代码——先让它"想清楚"再动手

直接要求 vs 推理先行

直接要求:

"帮我实现 eval 方法"

→ AI 可能漏掉边界情况

推理先行:

"在实现 eval 前, 先分析:

1. 需要处理哪几种 LObject 类型?
2. 可能的边界情况有哪些?
3. 给出实现步骤后, 再写代码。"

任务分解

"这个 Evaluator 超过 100 行,
请拆分为以下步骤分别实现:

Step 1: 实现 eval 核心逻辑

Step 2: 实现算术运算 +, -, *, /

Step 3: 实现 cons/car/cdr

Step 4: 实现 evalToString"

每个 AI 响应控制在 100 行代码以内,
超出则主动要求拆分。

项目级技巧③ — 跨文件感知与回归防护

修改前分析影响面，修改后触发自我审查

修改前：影响面分析

"我要修改 LList 的 content 字段访问方式。
请先列出：哪些文件/方法访问了 content？
修改后可能影响哪些功能？"

修改后：自我 Code Review

"以高级工程师视角审查刚才的代码：
- 是否破坏了现有接口？
- 是否引入了新的边界情况？
- 是否需要更新调用方？"

在我们的作业中，修改 LList 会影响 Parser、Evaluator、所有测试文件——不做影响面分析很容易"改一处，崩三处"。

项目级技巧④ — Prompt 四要素

完整的项目级 Prompt 包含四个要素：

要素	说明	示例
背景	为什么做？目标是什么？	课程大作业，实现 Lambda Calculus 解释器
需求	具体功能需求	实现 eval 方法，支持 +,-,*,/,cons
技术栈	语言/框架/架构	Java 11 + Maven, Lexer→Parser→Eval 流水线
约束	编码标准/限制	不修改已有 Token 类，JUnit 4 测试

四要素缺一不可——缺少约束，AI 会生成"能跑但不符合项目规范"的代码。

AI 辅助代码质量 — Debug 四步法

步骤	提示词重点	说明
1. 提供完整上下文	粘贴完整 stack trace + 相关代码	上下文越全，诊断越准
2. 分析根因	"先给出分析， 不要修改代码 "	先理解问题，再动手
3. 最小化修复	"只修改 X 方法，不改其他"	控制改动范围
4. 验证修复	"生成正常路径 + 异常路径测试用例"	确保修复有效

实战示例

"以下是 eval 方法抛出的 NullPointerException:
[粘贴 stack trace]

请先分析根因，不要修改代码。
然后给出最小修复方案，只修改 eval 方法。
最后生成 2 个测试用例验证修复。"

直接让 AI "修复 Bug" 往往改动范围太大——分步走，你才能理解根因。

AI 辅助代码质量 — 测试生成与审查

AI 生成测试用例

"为以下 eval 方法生成 JUnit 4 测试，
每个方法至少包含：

- 1. 正常路径
- 2. 空结果 / 边界条件
- 3. 异常路径

[粘贴 eval 方法]"

AI 生成示例：

```
@Test
public void testEvalAtom() { ... }
@Test
public void testEvalEmptyList() { ... }
@Test(expected = RuntimeException.class)
public void testEvalUnknownSymbol() { ... }
```

AI 辅助代码审查

"以高级工程师视角审查以下代码，
重点检查：

- 1. 安全性：是否有空指针风险？
- 2. 性能：是否有不必要的循环？
- 3. 可维护性：命名是否清晰？
- 4. 边界条件：空值、越界是否处理？

[粘贴代码]"

AI 审查的局限

AI 擅长	AI 不擅长
语法/拼写错误	业务逻辑正确性
常见安全漏洞	需求合理性
命名规范	架构层面判断

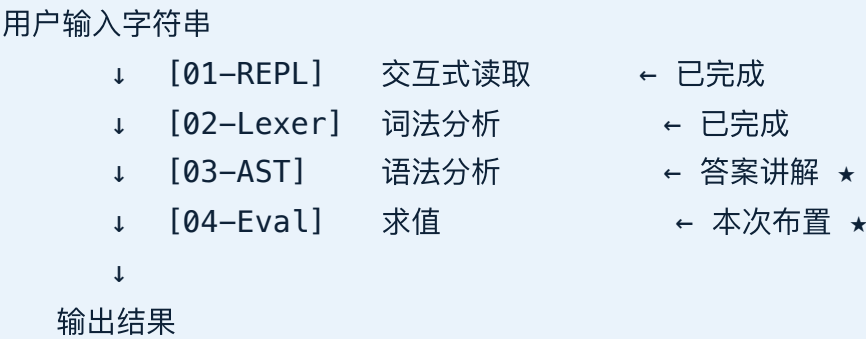
AI 是审查的补充，不是替代。

大作业讲解

Project 03 AST 参考答案 · Project 04 Eval 布置

大作业进度回顾

解释器流水线



完成情况

Project	状态	核心内容
01 REPL	已完成	交互式循环
02 Lexer	已完成	字符串 → Token 列表
03 AST	答案讲解	Token 列表 → 语法树
04 Eval	本次布置	语法树 → 求值结果

截止 Project 04 截止时间：下次课开始前

Project 03 答案 — 文件结构总览

```
03-ast-ans/
├─ src/main/java/edu/nju/software/
│   │   └─ stream/
│   │       │   └─ CharLookaheadStream.java    ← 字符流 (复用)
│   │       └─ TokenReadStream.java            ← Token 流 (新增)
│   └─ token/
│       │   └─ Token.java, Lexer.java          ← 词法分析 (复用)
│       └─ IntegerToken / StringToken / SymbolToken / LeftParen / RightParen
│   └─ ast/
│       │   └─ LObject.java    ← 抽象基类
│       │   └─ LInteger.java   ← 整数节点
│       │   └─ LString.java    ← 字符串节点
│       │   └─ LSymbol.java    ← 符号节点
│       │   └─ LList.java      ← 列表节点 (含子节点)
│       └─ Parser.java        ← 解析器 (核心 + 两个扩展)
└─ src/test/java/
    │   └─ AstTest.java        ← 公开测试 (16 个)
    └─ PrivateAstTest.java    ← 暗测试 (20 个)
```

Project 03 答案 — TokenReadStream

为什么需要 TokenReadStream?

Lexer 输出 `ArrayList<Token>`，但 Parser 需要逐个消费 Token:

- `peek()` — 预览当前 Token (不移动指针)
- `next()` — 读取当前 Token 并前进

```
public class TokenReadStream {
    private final ArrayList<Token> tokens;
    private int pos = 0;

    public Token peek() {
        if (pos >= tokens.size())
            return null;
        return tokens.get(pos);
    }

    public Token next() {
        Token t = peek();
        pos++;
        return t;
    }
}
```

与 CharLookaheadStream 对比

特征	CharLookahead	TokenRead
数据源	字符串	Token 列表
预览	<code>lookahead(n)</code>	<code>peek()</code>
消费	<code>consume()</code>	<code>next()</code>
EOF	<code>eof()</code>	<code>peek() == null</code>

设计模式

两个流类都遵循迭代器模式:

- 持有一个位置指针 `pos`
- 提供预览和消费两种操作
- 调用者不需要关心内部实现

核心理解: Lexer 用 `CharLookaheadStream` 逐字符读, Parser 用 `TokenReadStream` 逐 Token 读——两层抽象, 各司其职。

Project 03 答案 — AST 类层次

类层次（继承）

LObject	← 抽象基类
├─ LInteger	← value: int
├─ LString	← value: String
├─ LSymbol	← name: String
└─ LList	← content: List<LObject>

共同特征

所有子类都实现了：

- `equals()` + `hashCode()`（值相等）
- `toString()`（可读输出）
- `public final` 字段（不可变）

LList 核心设计

```
public class LList extends LObject {
    public final List<LObject> content;
    public LList(List<LObject> lst) {
        this.content = Collections
            .unmodifiableList(new ArrayList<>(lst));
    }
    // 空列表构造器
    public LList() {
        this.content = Collections.emptyList();
    }
    // 便捷方法
    public int size() { return content.size(); }
    public LObject get(int i) {
        return content.get(i);
    }
}
```

`unmodifiableList`：创建后不可修改——
不可变性让代码更安全、更易推理。

Project 03 答案 — Parser 入口与分发

```
public class Parser {  
    // 入口: 创建 TokenReadStream, 调用 parseForm  
    public static LObject parse(ArrayList<Token> tokens) {  
        return parseForm(new TokenReadStream(tokens));  
    }  
  
    // 分发: 左括号 → parseList, 否则 → parseAtom  
    static LObject parseForm(TokenReadStream rdr) {  
        Token token = rdr.peek();  
        if (token instanceof LeftParen) return parseList(rdr);  
        else return parseAtom(rdr);  
    }  
}
```

设计思路

- `parse` 是公共入口, 包装 `TokenReadStream`
- `parseForm` 是分发器: 看一眼 (peek) 就决定走哪条路
- 左括号 → 列表解析; 其他 → 原子解析

递归下降解析的核心: `parseForm` 根据下一个 Token 分发到不同的解析方法。

Project 03 答案 — Parser parseList 与 parseAtom

parseList — 解析列表

```
static LObject parseList(TokenReadStream rdr) {
    rdr.next(); // 消耗 '('
    ArrayList<LObject> lst = new ArrayList<>();
    while (rdr.peek() != null
           && !(rdr.peek() instanceof RightParen))
        lst.add(parseForm(rdr)); // 递归!
    rdr.next(); // 消耗 ')'
    return new LList(lst);
}
```

核心：循环中递归调用 `parseForm`，
处理嵌套表达式 `(+ (* 2 3) 4)`。

parseAtom — 解析原子

```
static LObject parseAtom(TokenReadStream rdr) {
    Token token = rdr.next();
    if (token instanceof IntegerToken)
        return new LInteger(
            ((IntegerToken) token).value);
    else if (token instanceof StringToken)
        return new LString(
            ((StringToken) token).value);
    else if (token instanceof SymbolToken)
        return new LSymbol(
            ((SymbolToken) token).name);
    throw new RuntimeException(
        "Unknown Token Type: " + token);
}
```

`instanceof` 分发：根据 Token 类型创建对应的 LObject 子类。

Project 03 答案 — Parser 执行轨迹

以 `"(+ 1 2)"` 为例，追踪整个解析过程：

```
Tokens: [ LeftParen SymbolToken(+) IntegerToken(1) IntegerToken(2) RightParen ]  
         ↑ pos=0
```

```
parse → parseForm → peek()='(' → parseList  
├─ next()           消耗 '(', pos=1  
├─ peek()='+'       → parseForm → parseAtom → LSymbol("+"), pos=2  
├─ peek()='1'       → parseForm → parseAtom → LInteger(1), pos=3  
├─ peek()='2'       → parseForm → parseAtom → LInteger(2), pos=4  
├─ peek()=' )'     → 退出循环  
└─ next()           消耗 ')', pos=5
```

```
结果: LList [ LSymbol("+"), LInteger(1), LInteger(2) ]
```

递归下降解析： `parseForm` 在 `parseList` 内部被递归调用——处理嵌套表达式 `(+ (* 2 3) 4)` 时，会自动递归进入内层列表。

Project 03 答案 — 扩展1 astToString

将 AST 节点转换回 Lisp 字符串:

```
public static String astToString(LObject ast) {
    if (ast instanceof LInteger)
        return Integer.toString(
            ((LInteger) ast).value);
    else if (ast instanceof LString)
        return "\"" + ((LString) ast).value
            + "\"";
    else if (ast instanceof LSymbol)
        return ((LSymbol) ast).name;
    else if (ast instanceof LList) {
        List<LObject> items =
            ((LList) ast).getContent();
        if (items.isEmpty()) return "()";
        StringBuilder sb = new StringBuilder("(");
        for (int i = 0; i < items.size(); i++) {
            if (i > 0) sb.append(" ");
            sb.append(astToString(items.get(i)));
        }
        sb.append(")");
        return sb.toString();
    }
    throw new RuntimeException("Unknown type");
}
```

转换规则

输入	输出
LInteger(42)	"42"
LString("hi")	"\"hi\""
LSymbol("+")	"+"
LList()	"()"
LList[+, 1, 2]	"(+ 1 2)"

核心模式

- instanceof 判断类型
- └ 原子 → 直接转换
 - └ LList → StringBuilder 拼接 + 递归处理子元素

这个模式在 `evalToString` (Project 04) 中会再次出现——
掌握一次 复用多次

Project 03 答案 — 扩展2 countAtoms

递归统计原子（非列表）节点总数：

```
public static int countAtoms(LObject ast) {
    if (ast instanceof LList) {
        int count = 0;
        for (LObject item :
            ((LList) ast).getContent()) {
            count += countAtoms(item); // 递归
        }
        return count;
    }
    return 1; // LInteger/LString/LSymbol 计 1
}
```

执行示例

```
countAtoms( parse("(+ (* 2 3) 4)") )

countAtoms(LSymbol("+"))      → 1
countAtoms(LList[*, 2, 3]) → 1+1+1 = 3
countAtoms(LInteger(4))      → 1
                               合计 = 5
```

递归思维

类型	返回
原子	1
空列表	0
非空列表	各子节点 countAtoms 之和

递归 = 信任子问题已解决。

Project 03 — 五个教学要点

#	要点	在代码中的体现
1	递归下降解析	<code>parseForm</code> → <code>parseList</code> → <code>parseForm</code> (循环递归)
2	<code>instanceof</code> 分发	<code>parseAtom</code> 根据 Token 类型创建不同 LObject
3	不可变设计	<code>LList.content</code> 用 <code>unmodifiableList</code> 包装
4	迭代器模式	<code>TokenReadStream</code> 的 <code>peek/next</code> 与 <code>CharLookaheadStream</code> 的 <code>lookahead/consume</code>
5	递归遍历 AST	<code>astToString</code> 和 <code>countAtoms</code> 都用递归处理嵌套结构

与后续 Project 的关系

Project 03 的模式	→ 在后续 Project 中复用
<code>instanceof</code> 分发	→ <code>eval</code> 中判断 LObject 类型
递归遍历 AST	→ <code>eval</code> 递归求值
StringBuilder 拼接	→ <code>evalToString</code>

掌握 Project 03 = 为 Project 04 打下基础——eval 的核心逻辑就是"递归遍历 AST + instanceof 分发"。

Project 04 布置

| Eval — 求值器

Project 04 — Eval 背景与目标

求值 (Evaluation)：遍历 AST，执行计算，得到最终结果。

```
输入字符串 "(+ 1 2)"
  ↓ Lexer
Token 列表 [( + 1 2 )]
  ↓ Parser
AST: LList [ LSymbol("+"), LInteger(1), LInteger(2) ]
  ↓ Evaluator (本次实现 ★)
结果: LInteger(3)
```

四条求值规则

规则	输入类型	行为
1	LInteger / LString	原子：直接返回自身
2	LSymbol	符号查找：在 globals 哈希表中查找
3	LList (空)	空列表：直接返回自身
4	LList (非空)	函数调用：对每个元素求值，第一个元素作为函数

Project 04 — 规则 4 详解：函数调用

规则 4 是最关键的规则——理解它就理解了整个 Evaluator。

```
eval( LList[LSymbol("+"), LInteger(1), LInteger(2)] )
```

Step 1: 对每个子元素调用 eval

```
eval(LSymbol("+")) → LFunction(+)      // 规则 2: 符号查找  
eval(LInteger(1)) → LInteger(1)        // 规则 1: 原子返回自身  
eval(LInteger(2)) → LInteger(2)        // 规则 1: 原子返回自身
```

Step 2: 组成新的求值后列表

```
LList[LFunction(+), LInteger(1), LInteger(2)]
```

Step 3: 第一个元素是函数, 调用 call

```
LFunction(+).call( LList[LFunction(+), LInteger(1), LInteger(2)] )  
→ 1 + 2 = LInteger(3)
```

注意: `call` 的参数是整个列表 (包含函数本身在 index 0), 所以操作数从 index 1 开始。

Project 04 — LFunction 函数对象

LFunction 类定义

```
public class LFunction extends LObject {
    public final int ary;          // 参数个数
    public final Function<LList, LObject>
        function;                  // Lambda 实现

    public LObject call(LList l) {
        // +1 因为 index 0 是函数本身
        if (l.content.size() != ary + 1)
            throw new IllegalArgumentException();
        return function.apply(l);
    }
}
```

内建函数注册示例（已提供的 +）

```
globals.put(new LSymbol("+"),
    new LFunction(2,
        list -> new LInteger(
            ((LInteger) list.content.get(1)).value
            + ((LInteger) list.content.get(2)).value
        )));
```

参数索引规则

索引	内容	示例 (+ 1 2)
get(0)	函数本身	LFunction(+)
get(1)	第一个操作数	LInteger(1)
get(2)	第二个操作数	LInteger(2)

ary = 2，call 时列表长度 = 2+1 = 3。

Project 04 — 你需要实现的 8 个 TODO

TODO	内容	难度	提示
TODO 1	减法 -	简单	参照 + 改运算符
TODO 2	乘法 *	简单	同上
TODO 3	除法 /	简单	整数除法
TODO 4	cons 函数	中等	新 ArrayList, 先 add 再 addAll
TODO 5	car 函数 (扩展2)	简单	取列表第一个元素
TODO 6	cdr 函数 (扩展2)	中等	subList(1, size) 包装为新 LList
TODO 7	eval 方法 (核心)	较难	instanceof 分发 + stream().map()
TODO 8	evalToString (扩展1)	中等	参照 astToString 的模式

TODO 7 核心伪代码

```
eval(obj):  
    if obj 是 LSymbol → globals.get(obj), 找不到则抛异常  
    if obj 是 LList → 空则返回自身, 非空则对每个元素 eval, 第一个作为函数 call  
    else → 返回自身 (原子)
```

Project 04 — cons / car / cdr 图解

cons — 在列表前端插入

```
(cons 3 nil)
→ LList(3)
```

```
(cons 5 (cons 3 nil))
→ LList(5, 3)
```

实现思路:

```
LObject toAdd = list.content.get(1);
LList oldList =
    (LList) list.content.get(2);
ArrayList<LObject> newList
    = new ArrayList<>();
newList.add(toAdd);
newList.addAll(oldList.content);
return new LList(newList);
```

car — 取列表头

```
(car (cons 1 (cons 2 nil)))
→ 1    (第一个元素)
```

提示: `content.get(0)`

cdr — 取列表尾

```
(cdr (cons 1 (cons 2 nil)))
→ (2)    (去掉第一个后的剩余)
```

提示: `subList(1, size)`

car/cdr 源自 Lisp 历史:

Contents of Address Register

Contents of Decrement Register

(IBM 704 机器指令)

Project 04 — 测试用例说明

运行测试

```
cd 04-eval
mvn test
```

基础测试

测试	输入	期望
testAtom	15	LInteger(15)
testEmpty	()	LList()
testAdd	(+ 1 2)	LInteger(3)
testSub	(- 5 3)	LInteger(2)
testMul	(* 4 3)	LInteger(12)
testDiv	(/ 7 2)	LInteger(3)
testNil	nil	LList()
testCons	(cons 3 nil)	LList(3)
testNested	(cons 5 (cons 3 nil))	LList(5,3)

扩展测试

测试	验证内容
test_ext1_integer	evalToString(42) → "42"
test_ext1_string	evalToString("hi") → "\"hi\""
test_ext1_empty_list	evalToString(()) → "()"
test_ext1_simple_list	列表转字符串
test_ext2_car	car 取列表头
test_ext2_cdr	cdr 取列表尾

全部绿色 = 作业完成

公开 16 个测试 + 暗测试若干；
通过率将记入平时分。

课后作业

必做

1. 完成 Project 04 Eval

提交到 SEECODER-Coder，截止下次课前。

8 个 TODO 都需完成：

`eval`、`-`、`*`、`/`、`cons`、`car`、`cdr`、`evalToString`

2. AI 辅助实验报告（一段话）

记录你用了哪种项目级 AI 技巧

（上下文注入 / 推理先行 / 跨文件感知），
AI 在哪里帮到了你，在哪里出错了

选做

- 用 AI 生成额外的测试用例来验证你的 eval 实现
- 尝试用 Debug 四步法让 AI 帮你定位一个 Bug
- 思考：如何让 eval 支持嵌套表达式
`(+ (- 3 1) (* 2 4))`？（提示：递归！）

预习

下周内容：继承与多态、HashMap

带着问题来：LObject 的类层次体现了
哪些 OOP 设计原则？

当堂练习

当堂练习 · 独立完成 · 25 分钟

练习 1 — 用 Stack 实现括号匹配

15 min 实现 `isBalanced` 方法，判断字符串中的括号是否匹配：

```
import java.util.Stack;

public class ParenMatch {
    // 支持三种括号: () [] {}
    // 规则: 每个右括号必须与最近的同类左括号匹配
    static boolean isBalanced(String s) {
        // 你的代码
    }

    public static void main(String[] args) {
        System.out.println(isBalanced("()")); // true
        System.out.println(isBalanced("([{}])")); // true
        System.out.println(isBalanced("]")); // false
        System.out.println(isBalanced("(")); // false
        System.out.println(isBalanced("")) // true
    }
}
```

提示：遇左括号 push，遇右括号检查 pop 是否匹配；最终栈为空 = 匹配成功。

练习 2 — 用 Queue 模拟任务调度

10 min 用 Queue 模拟轮转调度:

```
import java.util.*;

public class TaskScheduler {
    // 每轮处理队首任务 timeSlice 秒
    // 未完成则放回队尾
    static void roundRobin(
        String[] names, int[] times,
        int timeSlice) {
        Queue<int[]> queue = new LinkedList<>();
        // 你的代码
    }

    public static void main(String[] args) {
        String[] names = {"A", "B", "C"};
        int[] times = {5, 3, 7};
        roundRobin(names, times, 3);
    }
}
```

期望输出

处理 A, 剩余 2 秒
处理 B, 完成!
处理 C, 剩余 4 秒
处理 A, 完成!
处理 C, 剩余 1 秒
处理 C, 完成!

提示

1. 将所有任务入队 (用 `int[]` 存 {索引, 剩余时间})
2. 循环 `poll` 队首任务
3. 剩余时间减 `timeSlice`
4. 若 > 0 , `offer` 放回队尾
5. 若 ≤ 0 , 打印"完成"

第五次编程实践课

软件工程与计算 I · 2026 · 南京大学

感谢聆听 · 有问题随时提问